

Clase 3: Diseño de Arquitecturas de Red Neuronal para PPO y SAC

Fase Práctica DRL - Tesis Portafolio

12 de abril de 2026

Introducción al Diseño de Arquitecturas

En esta tercera fase de estudio, dejaremos de lado las ecuaciones del Entorno (MDP) para enfocarnos exclusivamente en la **Ingeniería de Cerebros Artificiales**. Responderemos a la pregunta: *¿Cómo estructuramos las cajas negras neuronales para que operen nuestro dinero de forma rentable?*

Todo nuestro diseño estará sometido estrictamente al documento maestro de Metodología de Implementación. Por instrucción inquebrantable de la tesis, ignoraremos por completo arquitecturas complejas y sobreparametrizadas (Redes Recurrentes LSTM o Convolucionales CNN).

Decisión Crítica: ¿Por qué obligatoriamente limitarnos a Perceptrones Multicapa (MLP)?

En un tribunal clásico de Machine Learning, podrían refutarte: *"La bolsa de valores es una serie de tiempo, ¿por qué no evaluar la historia de los retornos con una memoria recurrente LSTM?"*

Tu defensa es infalible porque nace de tu diseño previo del Entorno: **Tú ya tienes un modelo HMM de regímenes**. Las LSTM son necesarias cuando le entregas al modelo precios crudos a ciegas y esperas que sea él quien memorice y descubra los regímenes. En nuestro caso, el *Estado* s_t que la red ingiere ya contiene probabilidades matemáticas destiladas por el HMM (bear/bull) junto con matrices condicionadas de μ_t y Σ_t . El caos temporal del mercado ya fue comprimido de forma perfecta en un **Proceso de Markov**.

Si le pusiéramos una red con memoria LSTM a observar probabilidades ya establecidas por un modelo de Markov temporal (HMM), crearíamos una iteración conflictiva y redundante de memoria sobre memoria. Adicionalmente, el documento matriz exige: *usar 2 a 3 capas intermedias de tamaño moderado (64-128 neuronas) para evitar sobreparametrizar*". Nuestro objetivo es lograr que el Agente asigne capital agresivamente sin hacer "overfitting" sobre el ruido bursátil pasado. De ahí nace el reinado del **Perceptrón Multicapa Estándar (MLP)**.

1. Arquitectura para PPO: El Perfil Conservador

Proximal Policy Optimization requiere extrema estabilidad porque optimiza usando *Clipping*. Para evitar que las señales de aprendizaje se estrellen, diseñaremos las dos Redes del algoritmo PPO **totalmente separadas e independientes**.

Dada la menor complejidad de entrenar $V(s)$ en lugar del agresivo $Q(s, a)$, PPO lucirá una red más ligera y de cortes monótonos con funciones **Tanh (Tangente Hiperbólica)**.

1.1. Actor de PPO (La Política π)

La red neuronal que "adivina" los pesos óptimos w_t .

- **Capa de Entrada (Input):** Vector escalar del Estado Bursátil de tamaño $|S|$.
- **Capa Oculta 1:** Diferenciable Densa de **128 Neuronas** con activación **Tanh**.
- **Capa Oculta 2:** Diferenciable Densa de **128 Neuronas** con activación **Tanh**.
- **Capa de Salida:** Arroja un Vector continuo de **Dimensiones** $|I|$ (SPX y Cripto).
- *Nota de Acción:* En la implementación estricta, la red no arroja los pesos crudos directamente. Arrojará un valor $\tilde{a}_t \in [-1, 1]$ que luego el entorno **Gymnasium** transformará usando el mecanismo $\max(0)$ y lo empujará al Símpex dictado por $\frac{\max(0, \tilde{a}_{i,t})}{\sum \max(0, \tilde{a}_{j,t})}$.
- **Desviación Estándar (σ):** En PPO, el nivel de varianza exploratoria de esta campana de Gauss no la decide la red neuronal. Es un parámetro del sistema que se inicializa explícitamente e **independiente del estado temporal**, enfriándose hacia la explotación durante el entrenamiento.

1.2. Crítico de PPO (V_θ)

La red neuronal tasadora que solo se asegura de entregar una métrica escalar del valor al Actor.

- **Capa de Entrada (Input):** Vector escalar del Estado Bursátil de tamaño $|S|$.
- **Capa Oculta 1:** Diferenciable Densa de **128 Neuronas** con activación **Tanh**.
- **Capa Oculta 2:** Diferenciable Densa de **128 Neuronas** con activación **Tanh**.
- **Capa de Salida:** Un único nodo escalar lineal (Densa tamaño 1). Sin función de activación, pues estimará el valor económico real en sumatoria.

2. Arquitectura para SAC: El Explorador Entrópico

Soft Actor-Critic debe domar el inmenso caos adicional que genera su *Entropía Maximizada*. Mientras PPO busca rutas seguras, SAC se nutre del azar y requiere Redes **desarrolladas mediante activaciones ReLU** (sin topes superiores), sumado a la construcción monumental estipulada de Redes Gemelas.

Dado que busca funciones mucho más profundas del paisaje de la matriz de covarianzas, respetando la estructura pero dando ancho al canal, escalamos sus nodos sutilmente al límite sano recomendado.

2.1. Actor de SAC (La Política π)

Actor con volatilidad dinámica reactiva. A diferencia de PPO, se condicionará al estado turbulento.

- **Capa de Entrada (Input):** Vector escalar del Estado Bursátil.
- **Capa Oculta 1:** Diferenciable Densa de **256 Neuronas** con activación **ReLU**.
- **Capa Oculta 2:** Diferenciable Densa de **256 Neuronas** con activación **ReLU**.
- **Capa Dual de Salida:** Aquí diverge brutalmente. Dado que tenemos 2 activos en juego (SPX y Cripto), esta última capa neuronal arroja **4 valores numéricos simultáneos** (salidas bifurcadas):

1. Densa con salida de 2 nodos para modelar la media determinista de cada activo: $(\mu_{SPX}, \mu_{Cripto})$.
2. Densa separada de 2 nodos, **totalmente calculada por la red**, para modelar sus respectivos logaritmos de volatilidad: $(\log \sigma_{SPX}, \log \sigma_{Cripto})$.

Consecuencia: Si la red, en base a su Input HMM, huele que estamos en pleno Crack Financiero Cripto, el Actor disparará por sí mismo su métrica de desviación dinámica (σ_{Cripto}) a niveles astronómicos, forzándose a jugar con fuego (exploración entrópica audaz) en es criptomoneda, sin afectar la pasividad exploratoria del activo SPX.

2.2. Críticos Duales de SAC (Twin Q-Networks)

Dado que esta arquitectura no calcula V , sino Soft $Q(s_t, a_t)$, sufre una patología severa: “Overestimation Bias” (La IA tiene falsas esperanzas crónicas de que los activos Cripto seguirán subiendo por entropía inflada). Para solucionarlo requerimos clonar el crítico (Mecanismo TD3).

- **Capa de Entrada (Input):** Vector del Estado S_t unido de bloque al Vector de la Acción Decidida A_t ($|S| + |A|$).
- **Arquitectura Gemela Paralela (TD3):** Se construyen estrictamente dos redes totalmente separadas operando al unísono. Pese a poseer la *misma estructura física*, ambas nacen con inicializaciones aleatorias de pesos neuronales totalmente distintas. Al procesar datos estocásticos desde orígenes distintos, obtendrán “opiniones” levemente diferentes:
 - **Crítico 1:** Densa(256,ReLU) $\rightarrow$ Densa(256,ReLU) $\rightarrow$ Nodo Escalar para Valor Q_1 .
 - **Crítico 2:** Densa(256,ReLU) $\rightarrow$ Densa(256,ReLU) $\rightarrow$ Nodo Escalar para Valor Q_2 .
- **La Condición de Mitigación al Sesgo:** ¿Por qué sirven 2 redes de lo mismo? En DRL, el ruido estadístico suele confundir a una Red solitaria haciéndola creer que acaba de descubrir oro ilusorio. Al tener 2 gemelos con pesos independientes, el motor SAC les obligará a debatir y, mediante Backpropagation, evaluará sus rendimientos forzando un $Q_{target} \approx \min(Q_1(s, a), Q_2(s, a))$. Al penalizarlos forzándolos a usar siempre la estimación más pesimista, la IA desactiva matemáticamente el mortífero sesgo *Overestimation Bias* y evita sobre-operar inversiones infladas por la estupidez transitoria.

3. Parámetros Fundamentales del Entrenamiento DRL

Para dejar la infraestructura neuronal sin cabos sueltos teóricos, es fundamental definir cómo se moverán esos inmensos bloques matemáticos durante las épocas de aprendizaje iterativo en Python:

- **Las Condiciones Iniciales de los Pesos:**
 - *Pesos de las Acciones Financieras del Portafolio (w_0):* Cada vez que la simulación inicie un nuevo episodio en Gymnasium, tu dinero nacerá típicamente en un portafolio equilibrado inicial uniforme (ej: 50 % SPX y 50 % Cripto) antes de que la IA opere.
 - *Pesos de las Redes Neuronales (θ):* Los tensores internos que multiplican matrices en la IA no parten de base monetaria, sino que utilizan un sistema de Inicialización Ortogonal Aleatoria; es decir, nacen como números distribuidos estocásticamente en un entorno nulo para asegurar que los gradientes no exploten en los milisegundos de arranque.
- **Backpropagation y Optimizador ADAM:**

- La red neuronal no corrige sus errores con simple Descenso de Gradiente (SGD). Usaremos obligatoriamente **ADAM (Adaptive Moment Estimation)**. Este algoritmo matemático estandarizado calcula tasas de aprendizaje individuales adaptables para cada peso midiendo el primer y segundo momento de las varianzas. Literalmente, ADAM es capaz de reaccionar absorbiendo el ruido errático colosal que introducen las series de tiempo financieras, dándonos el entrenamiento más prístino posible en Deep Learning tabular.

■ La Fijación de la Entropía (α) en SAC:

- El parámetro de temperatura entrópica de la ecuación, α , es el multiplicador que define cuánto bono monetario le inyectamos a SAC por jugar de manera atrevida. En tu Tesis, dejaremos que ese valor inicie dinámico marcándolo como `.auto`. Esto desencadena una optimización de Backpropagation secundaria puramente matemática que sintoniza el multiplicador sin ayuda del humano. El SAC premiará locuras altísimas en sus primeras iteraciones y enfriará su α progresivamente apenas encuentre una política financiera seria y rentable, dirigiéndose numéricamente a una entropía objetivo estipulada heurísticamente en $-\dim(\mathcal{A})$ (en tu caso sería -2).

4. Esquemas Estructurales y Tabla Resumen

A continuación se plasma de manera visual el mapeo estructural de los cerebros artificiales que codificaremos para el portafolio.

4.1. Esquema de Flujo para PPO

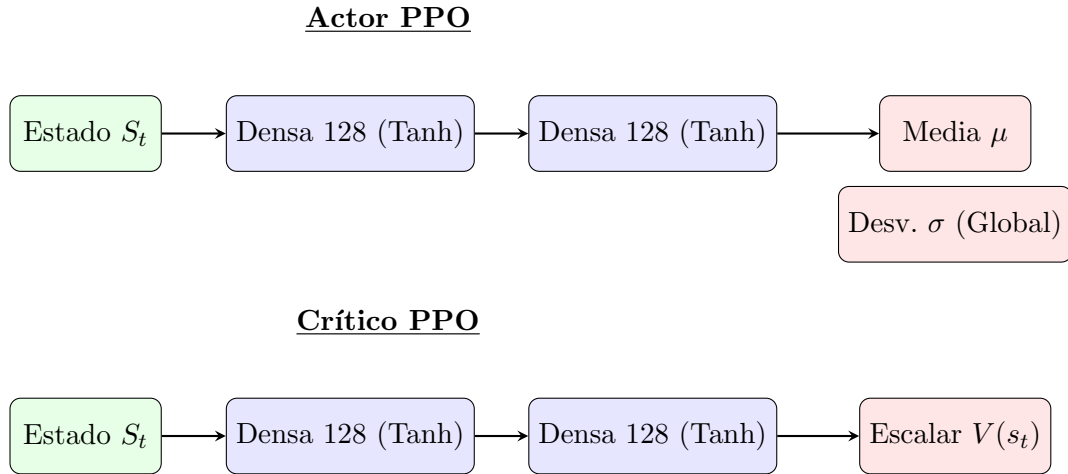


Figura 1: Diseño Arquitectónico de las Redes Separadas para Proximal Policy Optimization.

4.2. Esquema de Flujo para SAC

4.3. Tabla de Comparativa Final

Esta tabla sirve como herramienta referencial veloz para la implementación en Python.

Con esto, finalizamos el ensamblaje estricto de las infraestructuras de nuestros modelos de Deep Learning, cumpliendo con la filosofía de robustez impuesta en el documento maestro DRL Portafolio. La próxima fase abarca desplegar estos arquitectos en los flujos de Python estructurando los ciclos cronológicos en Gymnasium a base de diagramas y pseudocódigos.

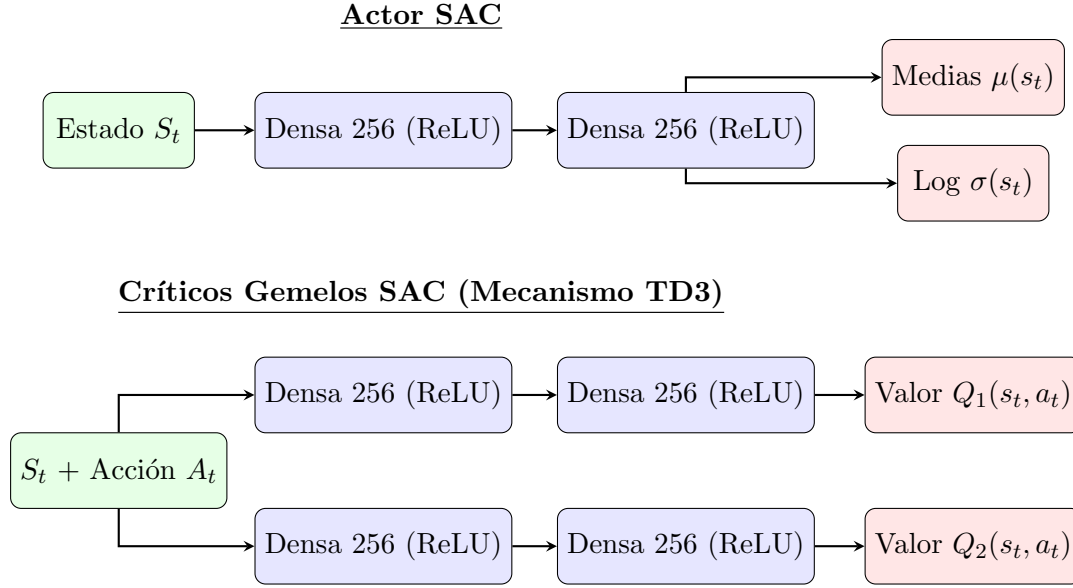


Figura 2: Diseño Arquitectónico de las Redes Entrópicas Soft Actor-Critic.

Propiedad Neural	PPO (Algoritmo Conservador)	SAC (Algoritmo Entrópico)
Tallas de Red (Actor)	MLP 128×128	MLP 256×256
Función Activación	Tanh (Monótona, fluida y acotada)	ReLU (Profunda y libre)
Nodos de Salida del Actor	2 Nodos: (Media μ_{SPX} , μ_{Cripto})	4 Nodos: (μ y $\log \sigma$ dinámicos)
Tallas de Red (Crítico)	Red Singular 128×128	Q-Networks Gemelos 256×256
Input Observado (Crítico)	Sólo el Estado actual (S_t)	El Estado (S_t) + La Acción (A_t)
Salida Final (Crítico)	Valor Generalizado $V(s_t)$	Mínimo protector: $\min(Q_1, Q_2)$
Optimizador Gradiente	ADAM	ADAM
Inicialización Interna	Matriz Ortogonal Estocástica	Matriz Ortogonal Estocástica

Cuadro 1: Síntesis de parámetros de Arquitectura de Red y Entrenamiento para el Entorno Financiero.